Stored-ID Unification
and Context-Sufficient Optimization

Mettapedia Working Notes

31 October 2026

Abstract

This note extends the named-shape-store theory with an executable stored-ID unifier and semantic contracts prompted by concrete runtime defects. The unifier retains terms as IDs, frames and lazy substitution closures; its readback agrees with the existing finite-tree unifier. Soundness, most-generality, finite completeness, logical rejection and exact resumption are proved in Lean. Further results justify free-frame occurs-check omission, shared skeleton alignment, contextual variable numbering, adaptive observers, captured readers and pure guard normalization. A fault-injection experiment also exposes a parallel-array capacity publication defect. Logical laws and native implementation guarantees remain separate throughout.

1 Retain information according to observations

For state s, retained information $k(s)$ and an observer $o(s)$, sufficiency is

$$k(s) = k(t) \quad \Longrightarrow \quad o(s) = o(t).$$

`PureGuardNormalForms.sufficient_factors` constructs an observer on the reachable keys under exactly this condition. Conversely, equal retained keys with different observable answers refute sufficiency. The observation may include more than a returned value: guards, effect order, multiplicity, completion status, live references and captured occurrence coordinates can all distinguish states.

This is an observer-relative information criterion. It is not a proof of globally optimal bit encoding, minimal allocation, or decidability of all program equivalences. A larger observer family can require a larger key.

2 Stored views and lazy substitutions

A faithful accessor exposes a stored term's next constructor and child IDs. Its source interpretation supplies the equation relating that single step to finite syntax. The native implementation must establish this accessor contract; the Lean development includes a concrete shared ID table as a nontrivial instance.

A view has one of three forms:

$$v ::= \mathrm{cell}(f, x) \mid \mathrm{stored}(i, f) \mid \mathrm{subst}(c, v, w).$$

Here f is a variable frame. The same stored variable under two frames denotes two distinct logic variables. Readback interprets the last constructor as single substitution:

$$[\![\mathrm{subst}(c, v, w)]\!] = [c \mapsto [\![v]\!]][\![w]\!].$$

Head exposure applies closures lazily: at a constructor it propagates them to the child views, rather than materializing the decoded term. The occurs walk uses this exposure operation. Its termination is proved by readback size, but its operational body does not compute readback.

`StoredIdViews.head_correct`, `occurs_correct` and `substitute_correct` establish the primitive simulation. The model uses binary constructor syntax, with constants and scoped variables; it does not claim a verified encoding of every native atom kind, list splice or grounded equality implementation.

3 The unifier and its completion boundary

The worklist algorithm exposes both operands, deletes equal variables or constants, decomposes aligned constructors, and installs lazy single substitutions after the occurs check. Returned bindings remain views, not decoded terms. For each allowance n,

$$\mathrm{solvedReadback}(\mathrm{run}(n, E)) = \mathrm{unifyFuel}(n, [\![E]\!]).$$

The two sides are independently recursive. The operational unifier never calls the reference unifier or readback.

The correspondence yields soundness and most-generality of every solved answer. Every solvable finite equation system succeeds with some finite allowance. The result type distinguishes *solved*, *conflict* and *paused*. A paused result retains the exact pending equations and ordered binding prefix. Splitting the allowance preserves the complete result:

$$\mathrm{run}(m + n, E) = \mathrm{resume}(n, \mathrm{run}(m, E)).$$

Solved and conflicting outcomes persist with additional allowance. Combining that persistence with finite completeness proves that conflict means no finite-term unifier exists; it is not budget exhaustion.

This is an immutable lazy-substitution reference, not a model of the C implementation's in-place binding cells. It proves no speedup or allocation bound: closure creation, repeated head exposure and worklist size still cost resources. Native mutation, allocation refusal and unsupported primitives retain separate refinement obligations.

4 Occurs-check omission and alignment

The free-frame shortcut requires both:

1. the target variable belongs to another frame;
2. no active binding has a left-hand variable in the stored term's frame.

The latter implies that the composed substitution fixes every variable of that frame. The framed term therefore cannot acquire an occurrence of the foreign target. A cached zero count justifies the shortcut only if it equals the count of active bindings. Append and rollback preserve that invariant in `StoredIdFrameConditions`; rollback restores the previous binding prefix and all frame counts. Distinct frames alone are insufficient when a binding redirects a variable into the target frame. An unbound frame alone is insufficient when the target belongs to that same frame.

`StoredIdAlignment` computes one query view per skeleton hole. Successful alignment has exactly the solutions of the full row equation, for every row filling and every subsequent shared

substitution. Thus one plan may be reused across a read, while retaining the original frames and lazy closures. A plan must not be reused after its captured prior bindings change.

The absence of an alignment certificate is not logical failure. In particular, a variable facing a constructor, including a variable at an inner head position, can unify with that constructor. The controls exhibit this case. The typed row representation enforces hole arity; a native packed-array adapter must enforce the corresponding boundary.

5 Contextual alpha numbering

Canonical variable numbers depend on the numbering context already accumulated for the whole result. A known cell keeps its number; an unseen cell extends the shared context. Numbering concatenated variable-occurrence streams agrees with numbering the second stream under the first stream's outgoing context.

`ContextualAlphaNumbering.check_exact` relates a streaming validator of stored ordinals to an independent number-producing traversal. A standalone canonical part can fail this validation inside a caller context. For example, after seeing cells a, b, the part b, a must use ordinals $1, 0$, not its standalone ordinals $0, 1$. Repeated parts retain repeated-variable relationships. The result covers the variable-numbering stream, not native traversal coverage or the full alpha codec.

6 Storage observations and mutation

The previous primitive bag refinement now lifts to finite programs whose next operation depends on previous answers. `ShapeObserverPrograms` preserves returned values, complete answer traces and final bags against an independent all-rows interpreter. Membership is derived from logical count, not record presence. Guards can observe earlier writes in the same pass, and duplicate insertions into a bag remain duplicate occurrences.

That bag theorem is not enough for a cursor already in progress. `CapturedRowReaders` distinguishes live revision validation from an owned captured prefix. Detaching before publication preserves the remaining rows, current position and subsequent occurrence results through arbitrary store edits. A revision-only edit can otherwise invalidate a reader despite unchanged row payloads. Epoch adoption is proved safe at the value level when the entire captured prefix is unchanged; bag equality alone does not suffice. Native index coordinates and pointer lifetimes need their own certificates.

An independent run of the saved S3g executable reproduced the shortest-path `revision-invalidated` failure; the all-rows control completed with the expected distances. The eager-mode dynamic-equation control also differed from all rows: its final result retained `(lg1)` rather than using the added equation. The latter implementation diagnosis remains open; the model does not label a particular cache as its confirmed cause.

7 Wider guard regions through proved normalization

Finite pure guard forms normalize to an ordered list of signed observations and literals. Empty alternatives disappear; negations compose without a fixed depth cap; metadata explicitly classified as inert disappears. Evaluation preserves the complete ordered list of Boolean answers, including duplicates. A singleton recognizer after normalization is therefore unchanged by an added inert empty alternative, an inert annotation, or double negation.

The proof connects to the existing ordered-guard continuation semantics, so replacing the pure provider preserves arbitrary consuming effects. Extraction of that pure form remains an obligation of the active source profile. An equation returning no answers can still perform effects while evaluating its arguments. In the existing operational semantics an effect followed by failure has no answers but changes the world. It is not removable as inert. Likewise, a type declaration cannot be erased without establishing that it does not change dispatch, argument behavior or failure in the selected profile.

These laws support wider regions than an equation-count or spelling test. They do not implement a new CeTTa recognizer or claim maximal admission of arbitrary semantically equivalent programs.

8 Refusal must leave reusable state valid

An allocator-failure probe against the inspected `term_molecule.c` exposed an independent defect. When cell-array growth succeeds but parallel frame-array growth fails, `mol_cell_of` returns decline with shared capacity 32 and frame-array capacity 16. A retry writes the seventeenth frame entry; AddressSanitizer reports a heap-buffer overflow. The probe injects one allocation failure and includes the actual implementation source; it does not modify the runtime worktree. This is failure-path evidence, not a claim that ordinary workloads trigger the failure.

`PairedArrayAdmission` formalizes the required shared-bound invariant:

$$\text{live} \leq \text{published} \leq \min(\text{valuesCapacity}, \text{framesCapacity}).$$

Private allocations may grow independently. Publishing the shared bound only after both successes preserves safe retry; refusal may retain a larger private allocation without advertising inaccessible slots. The model proves safe index admission and includes the failing $16 \to 32$ state as a counterexample. It is not a C patch or a proof of native allocator behavior.

9 Scope and qualification

The nine new modules under `Mettapedia/Machines/` are `StoredIdViews`, `StoredIdUnification`, `StoredIdFrameConditions`, `StoredIdAlignment`, `ShapeObserverPrograms`, `CapturedRowReaders`, `PureGuardNormalForms`, `ContextualAlphaNumbering` and `PairedArrayAdmission`. `StoredIdOptimizationAudit` inventories their module-owned mathematical declarations and transitive axiom dependencies. No admissions, added axioms or native decision proofs are used. Compiler runtime counterparts are inventoried separately from mathematical bodies.

The immediate native obligations are faithful stored-head access, shared scopes and numbering, complete observer support, correct capture/publication, failure-state validity and profile-qualified source normalization. Guard-aware aggregation and effect batching require complete consumer support and the authored order; no general commutativity or idempotence is assumed. The broader transducer/CTL, decision-diagram, higher-derivative and optimal-cut packages remain separate work.